

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «Интеллектуальный анализ данных»

Тема: «Предобучение нейронных сетей с использованием автоэнкодерного подхода»

Выполнила:

Студент 4 курса

Группы ИИ-26

Карпович К.А.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: научиться осуществлять предобучение нейронных сетей с помощью автоэнкодерного подхода

Общее задание

1. Взять за основу любую сверточную или полносвязную архитектуру с количеством слоев более 3. Осуществить ее обучение (без предобучения) в соответствии с вариантом задания. Получить оценку эффективности модели, используя метрики, специфичные для решаемой задачи (например, MAPE – для регрессионной задачи или F1/Confusion matrix для классификационной).
2. Выполнить обучение с предобучением, используя автоэнкодерный подход, алгоритм которого изложен в лекции. Условие останова (например, по количеству эпох) при обучении отдельных слоев с использованием автоэнкодера выбрать самостоятельно.
3. Сравнить результаты, полученные при обучении с/без предобучения, сделать выводы.
4. Выполните пункты 1-3 для датасетов из ЛР 2.
5. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№	Выборка	Тип задачи	Целевая переменная
6	https://archive.ics.uci.edu/dataset/265/physicochemical+properties+of+protein+tertiary+structure	регрессия	RMSD

Код программы:

```
import copy
import os
import time

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
from sklearn.datasets import load_breast_cancer
from sklearn.metrics import (accuracy_score, confusion_matrix, f1_score,
                             mean_absolute_error, mean_squared_error,
                             precision_score, r2_score, recall_score,
                             ConfusionMatrixDisplay)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

CASPath = "CASPath.csv"
OUT_DIR = "results"
SHOW_PLOTS = True

SEEDS = [0, 1, 2, 3, 4]
BATCH = 256

AE_EPOCHS = 30
AE_LR = 1e-3
HEAD_EPOCHS = 30

FT_LR = 1e-3

MAPE_MIN_Y = 1.0

DATASETS = {
    "CASPath (RMSD, регрессия)": dict(task="regression", hidden=[64, 32, 16, 8], batch=256, epochs=60, fracs=[1.0, 0.05]),
    "WDBC (рак груди, классификация)": dict(task="classification", hidden=[64, 32, 16, 8], batch=32, epochs=100, fracs=[1.0, 0.25]),
}
```

```
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
def load_casp():
    if os.path.exists(CASP_PATH):
        df = pd.read_csv(CASP_PATH)
    else:
        try:
            from ucimlrepo import fetch_ucirepo
            ds = fetch_ucirepo(id=265)
            df = pd.concat([ds.data.targets, ds.data.features], axis=1)
        except Exception as e:
            raise FileNotFoundError(
                f"Не найден {CASP_PATH}. Скачайте датасет с "
                "https://archive.ics.uci.edu/dataset/265 и положите CASP.csv рядом со скриптом."
            ) from e
    X = df.drop(columns=["RMSD"]).values.astype(np.float32)
    y = df["RMSD"].values.astype(np.float32)
    return X, y, None
```

```
def load_wdbc():
    data = load_breast_cancer()
    return data.data.astype(np.float32), data.target.astype(np.float32), list(data.target_names)
```

```
class MLP(nn.Module):

    def __init__(self, dims):
        super().__init__()
        self.layers = nn.ModuleList(nn.Linear(a, b) for a, b in zip(dims[:-1], dims[1:]))

    def features(self, x):
        for layer in self.layers[:-1]:
            x = torch.relu(layer(x))
        return x

    def forward(self, x):
        return self.layers[-1](self.features(x))
```

```
def batches(n, bs, gen):
    perm = torch.randperm(n, generator=gen)
    for i in range(0, n, bs):
        yield perm[i:i + bs].to(DEVICE)
```

```
def sup_loss(task, out, target):
    out = out.squeeze(1)
    if task == "regression":
        return F.mse_loss(out, target)
    return F.binary_cross_entropy_with_logits(out, target)
```

```
def pretrain_with_autoencoders(model, X, gen):
    ae_history, cur = [], X
    for k, layer in enumerate(model.layers[:-1]):
        decoder = nn.Linear(layer.out_features, layer.in_features).to(DEVICE)
        opt = torch.optim.Adam(list(layer.parameters()) + list(decoder.parameters()), lr=AE_LR)
        losses = []
        for _ in range(AE_EPOCHS):
            total = 0.0
            for idx in batches(len(cur), BATCH, gen):
                xb = cur[idx]
                rec = decoder(torch.relu(layer(xb)))
                loss = F.mse_loss(rec, xb)
                opt.zero_grad()
                loss.backward()
            opt.step()
```

```

        total += loss.item() * len(idx)
    losses.append(total / len(cur))
    ae_history.append(losses)
    with torch.no_grad():
        cur = torch.relu(layer(cur))
    return ae_history

def train_last_layer(model, X, y, task, gen):
    with torch.no_grad():
        feats = model.features(X)
    head = model.layers[-1]
    opt = torch.optim.Adam(head.parameters(), lr=FT_LR)
    for _ in range(HEAD_EPOCHS):
        for idx in batches(len(feats), BATCH, gen):
            loss = sup_loss(task, head(feats[idx]), y[idx])
            opt.zero_grad()
            loss.backward()
            opt.step()

@torch.no_grad()
def predict_raw(model, X):
    model.eval()
    out = model(X).squeeze(1).cpu().numpy()
    model.train()
    return out

def train_supervised(model, Xtr, ytr, Xte, yte, task, epochs, gen):
    opt = torch.optim.Adam(model.parameters(), lr=FT_LR)
    hist = {"train": [], "test": []}
    for _ in range(epochs):
        total = 0.0
        for idx in batches(len(Xtr), BATCH, gen):
            loss = sup_loss(task, model(Xtr[idx]), ytr[idx])
            opt.zero_grad()
            loss.backward()
            opt.step()
            total += loss.item() * len(idx)
        hist["train"].append(total / len(Xtr))
        with torch.no_grad():
            hist["test"].append(sup_loss(task, model(Xte), yte).item())
    return hist

def regression_metrics(y_true, y_pred):
    mask = y_true >= MAPE_MIN_Y
    mape = np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100
    return {"MAE": mean_absolute_error(y_true, y_pred),
            "RMSE": np.sqrt(mean_squared_error(y_true, y_pred)),
            "R2": r2_score(y_true, y_pred),
            "MAPE, %": mape}

def classification_metrics(y_true, prob):
    pred = (prob >= 0.5).astype(int)
    return {"Accuracy": accuracy_score(y_true, pred),
            "Precision": precision_score(y_true, pred, zero_division=0),
            "Recall": recall_score(y_true, pred, zero_division=0),
            "F1": f1_score(y_true, pred, zero_division=0)}

def evaluate(model, X, y_true, task, y_scaler):
    raw = predict_raw(model, X)
    if task == "regression":
        pred = y_scaler.inverse_transform(raw.reshape(-1, 1)).ravel()
        return regression_metrics(y_true, pred), pred
    prob = 1 / (1 + np.exp(-np.clip(raw, -50, 50)))

```

```

return classification_metrics(y_true, prob), prob

def run_seed(X_all, y_all, task, hidden, epochs, frac, seed):
    strat = y_all if task == "classification" else None
    Xtr, Xte, ytr, yte = train_test_split(X_all, y_all, test_size=0.2, random_state=seed, stratify=strat)
    if frac < 1.0:
        keep = np.random.RandomState(seed).permutation(len(Xtr))[:max(int(len(Xtr) * frac), 20)]
        Xtr, ytr = Xtr[keep], ytr[keep]

    x_scaler = StandardScaler().fit(Xtr)
    Xtr_s, Xte_s = x_scaler.transform(Xtr), x_scaler.transform(Xte)
    y_scaler = None
    ytr_s, yte_s = ytr, yte
    if task == "regression":
        y_scaler = StandardScaler().fit(ytr.reshape(-1, 1))
        ytr_s = y_scaler.transform(ytr.reshape(-1, 1)).ravel()
        yte_s = y_scaler.transform(yte.reshape(-1, 1)).ravel()

    t = lambda a: torch.tensor(np.asarray(a), dtype=torch.float32, device=DEVICE)
    Xtr_t, Xte_t, ytr_t, yte_t = t(Xtr_s), t(Xte_s), t(ytr_s), t(yte_s)

    dims = [X_all.shape[1]] + hidden + [1]
    torch.manual_seed(seed)
    base = MLP(dims).to(DEVICE)
    pre = copy.deepcopy(base)

    hist_base = train_supervised(base, Xtr_t, ytr_t, Xte_t, yte_t, task, epochs, torch.Generator().manual_seed(seed))
    m_base, out_base = evaluate(base, Xte_t, yte_t, task, y_scaler)

    gen = torch.Generator().manual_seed(seed)
    ae_hist = pretrain_with_autoencoders(pre, Xtr_t, gen)
    train_last_layer(pre, Xtr_t, ytr_t, task, gen)
    m_only, _ = evaluate(pre, Xte_t, yte_t, task, y_scaler)
    hist_pre = train_supervised(pre, Xtr_t, ytr_t, Xte_t, yte_t, task, epochs, gen)
    m_pre, out_pre = evaluate(pre, Xte_t, yte_t, task, y_scaler)

    return dict(metrics={"Без предобучения": m_base, "Только предобучение (без дообучения)": m_only,
                        "С предобучением + дообучение": m_pre},
              hist_base=hist_base, hist_pre=hist_pre, ae_hist=ae_hist,
              y_true=yte, out_base=out_base, out_pre=out_pre)

def make_figure(name, frac, task, runs, class_names, path):
    first = runs[0]
    plt.rcParams.update({"font.size": 11})
    fig, axes = plt.subplots(2, 2, figsize=(13, 10), layout="constrained")
    ax = axes.ravel()

    for k, losses in enumerate(first["ae_hist"]):
        ax[0].plot(losses, label=f"Слой {k + 1}")
    ax[0].set_title("Предобучение: потери реконструкции (MSE)")
    ax[0].set_xlabel("Эпоха")
    ax[0].set_ylabel("MSE")
    ax[0].legend(title="Автоэнкодер")
    ax[0].grid(alpha=0.3)

    for key, label in (("hist_base", "Без предобучения"), ("hist_pre", "С предобучением")):
        ax[1].plot(np.mean([r[key]["test"] for r in runs], axis=0), label=label)
    ax[1].set_title("Ошибка на тесте (среднее по запускам)")
    ax[1].set_xlabel("Эпоха")
    ax[1].set_ylabel("MSE (норм.)" if task == "regression" else "BCE")
    ax[1].legend()
    ax[1].grid(alpha=0.3)

    for a, key, title in ((ax[2], "out_base", "Без предобучения"), (ax[3], "out_pre", "С предобучением")):
        if task == "regression":
            n = min(3000, len(first["y_true"]))
            a.scatter(first["y_true"][:n], first[key][:n], s=5, alpha=0.4)

```

```

lim = [0, max(first["y_true"].max(), first[key].max())]
a.plot(lim, lim, "r--")
a.set_xlabel("Истинный RMSD")
a.set_ylabel("Предсказанный RMSD")
a.set_title(f'{title}: прогноз и факт')
else:
    cm = confusion_matrix(first["y_true"], (first[key] >= 0.5).astype(int))
    ConfusionMatrixDisplay(cm, display_labels=class_names).plot(ax=a, colorbar=False)
    a.set_xlabel("Предсказанный класс")
    a.set_ylabel("Истинный класс")
    a.set_title(f'{title}: матрица ошибок')
fig.suptitle(f'{name} \nдоля обучающей выборки = {frac}', fontsize=14)
fig.savefig(path, dpi=150)

def main():
    os.makedirs(OUT_DIR, exist_ok=True)
    print(f'Устройство: {DEVICE}')
    loaders = {"CASP (RMSD, регрессия)": load_casp, "WDBC (рак груди, классификация)": load_wdbc}
    rows = []

    for name, cfg in DATASETS.items():
        X, y, class_names = loaders[name]()
        task = cfg["task"]
        global BATCH
        BATCH = cfg["batch"]
        print(f'\n{'=' * 70} \n{name}: {X.shape[0]} объектов, {X.shape[1]} признаков")
        print(f'Архитектура: {[X.shape[1] + cfg["hidden"] + [1]}')
        for frac in cfg["fracs"]:
            runs = []
            t0 = time.time()
            for seed in SEEDS:
                r = run_seed(X, y, task, cfg["hidden"], cfg["epochs"], frac, seed)
                runs.append(r)
            for method, m in r["metrics"].items():
                rows.append({"Датасет": name, "Доля train": frac, "Метод": method, "seed": seed, **m})
            print(f'доля={frac}, seed={seed} готово ({time.time() - t0:.0f} c)')
            make_figure(name, frac, task, runs, class_names,
                        os.path.join(OUT_DIR, f'{name.split()[0]}_{frac}{frac}.png'))

    df = pd.DataFrame(rows)
    df.to_csv(os.path.join(OUT_DIR, "all_runs.csv"), index=False, encoding="utf-8-sig")
    metric_cols = [c for c in df.columns if c not in ("Датасет", "Доля train", "Метод", "seed")]
    summary = df.groupby(["Датасет", "Доля train", "Метод", sort=False])[metric_cols].agg(["mean", "std"])
    summary.to_csv(os.path.join(OUT_DIR, "summary.csv"), encoding="utf-8-sig")

    pd.set_option("display.width", 250, "display.max_columns", 50)
    print("\n" + "=" * 70 + "\nИТОГОВЫЕ РЕЗУЛЬТАТЫ (mean по запускам; std в summary.csv)\n" + "=" * 70)
    for (ds, frac), g in df.groupby(["Датасет", "Доля train"], sort=False):
        print(f'\n{ds}, доля обучающей выборки = {frac}')
        cols = [c for c in metric_cols if g[c].notna().any()]
        print(g.groupby("Метод", sort=False)[cols].mean().round(4).to_string())

    print(f'\nГрафики и таблицы сохранены в папку '{OUT_DIR}')
    if SHOW_PLOTS:
        plt.show()

if __name__ == "__main__":
    main()

```

Результат программы:

CASP (RMSD, регрессия): 45730 объектов, 9 признаков
Архитектура: [9, 64, 32, 16, 8, 1]
доля=1.0, seed=0 готово (29 с)
доля=1.0, seed=1 готово (57 с)
доля=1.0, seed=2 готово (85 с)
доля=1.0, seed=3 готово (113 с)
доля=1.0, seed=4 готово (141 с)
доля=0.05, seed=0 готово (2 с)
доля=0.05, seed=1 готово (3 с)
доля=0.05, seed=2 готово (5 с)
доля=0.05, seed=3 готово (7 с)
доля=0.05, seed=4 готово (8 с)

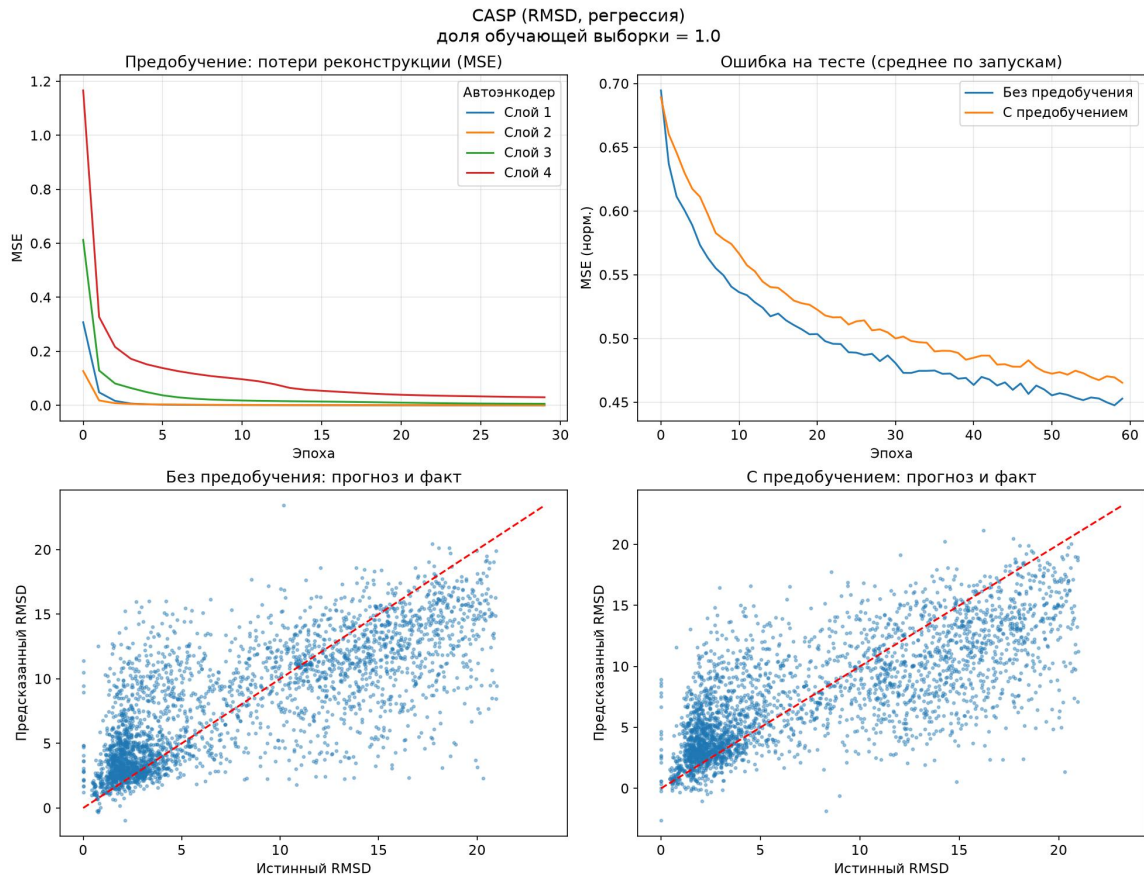
=====

WDBC (рак груди, классификация): 569 объектов, 30 признаков
Архитектура: [30, 64, 32, 16, 8, 1]
доля=1.0, seed=0 готово (4 с)
доля=1.0, seed=1 готово (8 с)
доля=1.0, seed=2 готово (12 с)
доля=1.0, seed=3 готово (16 с)
доля=1.0, seed=4 готово (20 с)
доля=0.25, seed=0 готово (1 с)
доля=0.25, seed=1 готово (2 с)
доля=0.25, seed=2 готово (3 с)
доля=0.25, seed=3 готово (4 с)
доля=0.25, seed=4 готово (5 с)

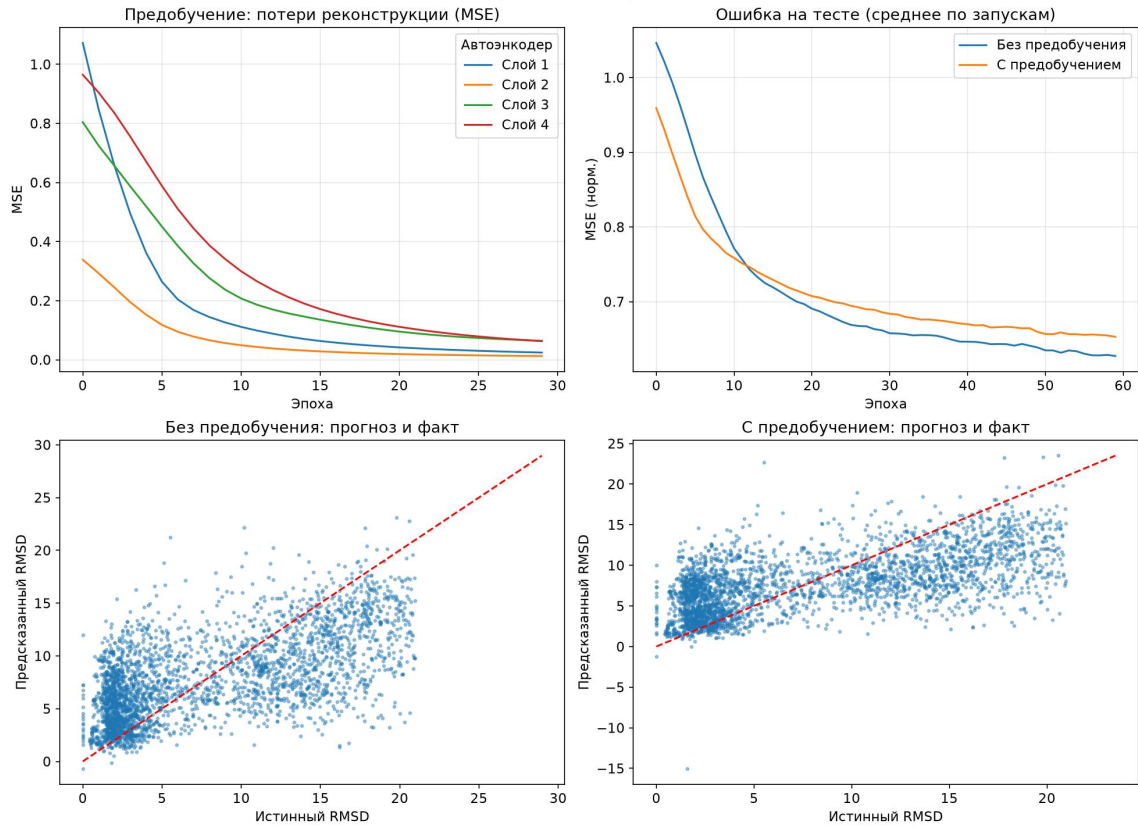
=====

ИТОГОВЫЕ РЕЗУЛЬТАТЫ (mean по запускам; std в summary.csv)

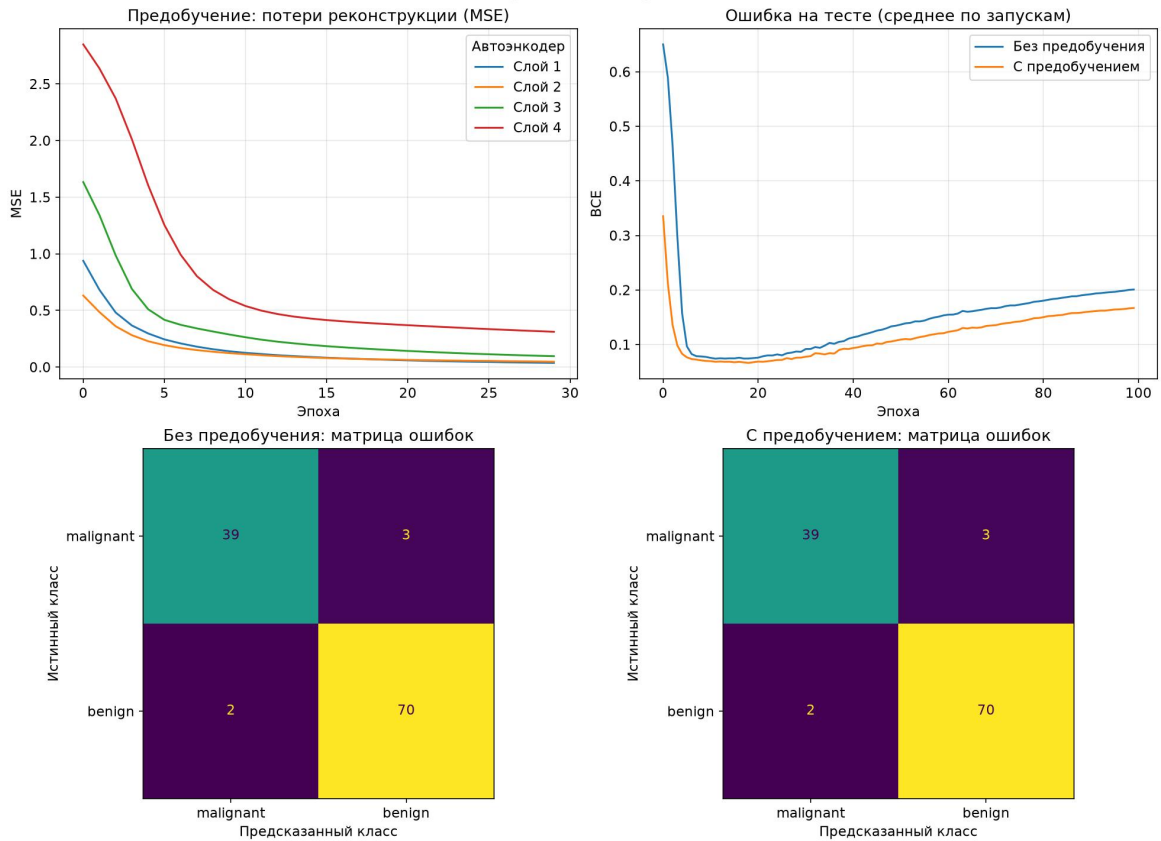
CASP (RMSD, регрессия), доля обучающей выборки = 1.0				
	MAE	RMSE	R2	MAPE, %
Метод				
Без предобучения	2.9935	4.1177	0.5461	63.9108
Только предобучение (без дообучения)	4.6526	5.4656	0.2002	113.6060
С предобучением + дообучение	3.1069	4.1743	0.5336	67.9738
CASP (RMSD, регрессия), доля обучающей выборки = 0.05				
	MAE	RMSE	R2	MAPE, %
Метод				
Без предобучения	3.8451	4.8410	0.3728	91.4554
Только предобучение (без дообучения)	5.4860	6.0915	0.0069	137.8876
С предобучением + дообучение	3.9883	4.9397	0.3469	97.1570
WDBC (рак груди, классификация), доля обучающей выборки = 1.0				
	Accuracy	Precision	Recall	F1
Метод				
Без предобучения	0.9754	0.9727	0.9889	0.9807
Только предобучение (без дообучения)	0.7982	0.7819	0.9694	0.8620
С предобучением + дообучение	0.9754	0.9727	0.9889	0.9807
WDBC (рак груди, классификация), доля обучающей выборки = 0.25				
	Accuracy	Precision	Recall	F1
Метод				
Без предобучения	0.9544	0.9645	0.9639	0.9638
Только предобучение (без дообучения)	0.6035	0.5416	0.7417	0.6219
С предобучением + дообучение	0.9561	0.9646	0.9667	0.9652
Графики и таблицы сохранены в папку 'results'				

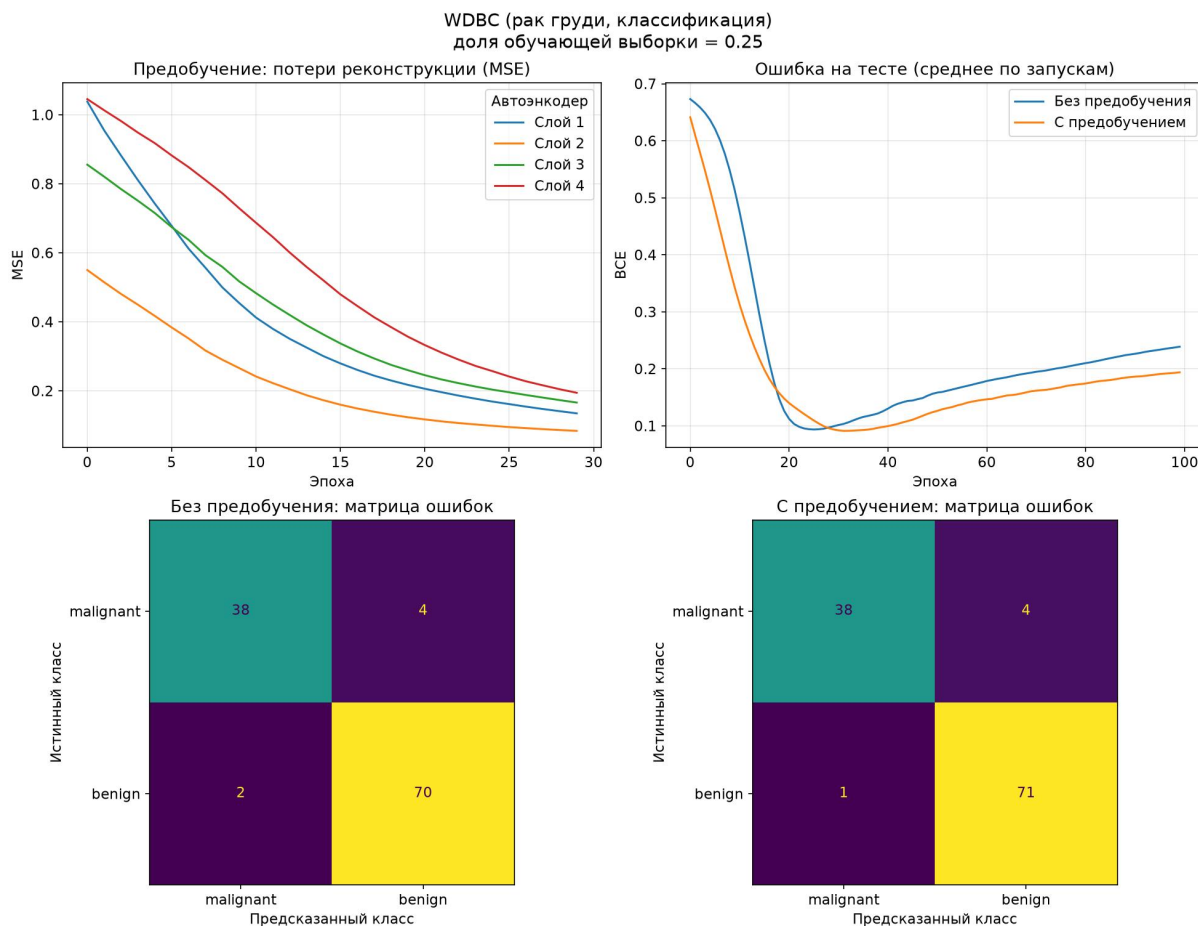


CASP (RMSD, регрессия)
доля обучающей выборки = 0.05



WDBC (рак груди, классификация)
доля обучающей выборки = 1.0





Выводы о проделанной работе: в работе исследовано автоэнкодерное (послойное) предобучение глубокой полносвязной сети с четырьмя скрытыми слоями на двух выборках: CASP (регрессия RMSD, 45 730 объектов, 9 признаков) и WDBC (из ЛР 2, классификация, 569 объектов, 30 признаков). Обе модели (с предобучением и без) стартовали с одинаковых весов, каждый эксперимент повторён на 5 разбиениях, результаты усреднены. Дополнительно проверены уменьшенные обучающие выборки (5% и 25%).

1. Автоэнкодеры всех слоёв обучились успешно: потери реконструкции монотонно снижаются.
2. **CASP.** Предобучение не улучшило, а слегка ухудшило качество: $R^2 = 0,534$ против $0,546$, $RMSE = 4,17$ против $4,12$, $MAPE = 68,0\%$ против $63,9\%$. На 5% выборки результат такой же ($R^2 = 0,347$ против $0,373$), ожидаемого выигрыша на малой выборке нет.
3. **WDBC.** На полной выборке метрики совпали (Accuracy = $97,54\%$, $F1 = 0,981$). На 25% выборки разница ($F1 = 0,965$ против $0,964$) составляет один объект в одном запуске и лежит в пределах разброса, поэтому значимой не является. Предобучение лишь ускоряет сходимость в первые эпохи.
4. Без дообучения предобученная сеть сильно уступает ($R^2 = 0,20$ на CASP, $F1 = 0,86$ на WDBC): автоэнкодеры не учитывают целевую переменную, поэтому дообучение с учителем обязательно.
5. Отсутствие выигрыша объясняется тем, что сеть небольшая и обучается алгоритмом Adam с ReLU, поэтому проблема исчезающего градиента почти не проявляется. В CASP всего 9 признаков, а автоэнкодеры глубоких слоёв почти идеально восстанавливают вход, не находя новых признаков. WDBC хорошо разделен, поэтому обе модели достигают предела качества.

Автоэнкодерное предобучение реализовано и работает, но для выбранных датасетов и архитектур не дало статистически значимого улучшения качества: на CASP оно незначительно ухудшило результат, на WDBC результаты совпали, а некоторое ускорение сходимости наблюдалось только на WDBC. Метод оправдан для более глубоких сетей и многомерных данных со сложной структурой (например, изображений), а для небольших табличных наборов данных достаточно обычного обучения с ReLU и Adam.